



Natural Language Processing Introduction

Advanced Software-Engineering
Dr. Harald Stein
Prof. Dr. Selcan Ipek-Ugay
WS 2026/27



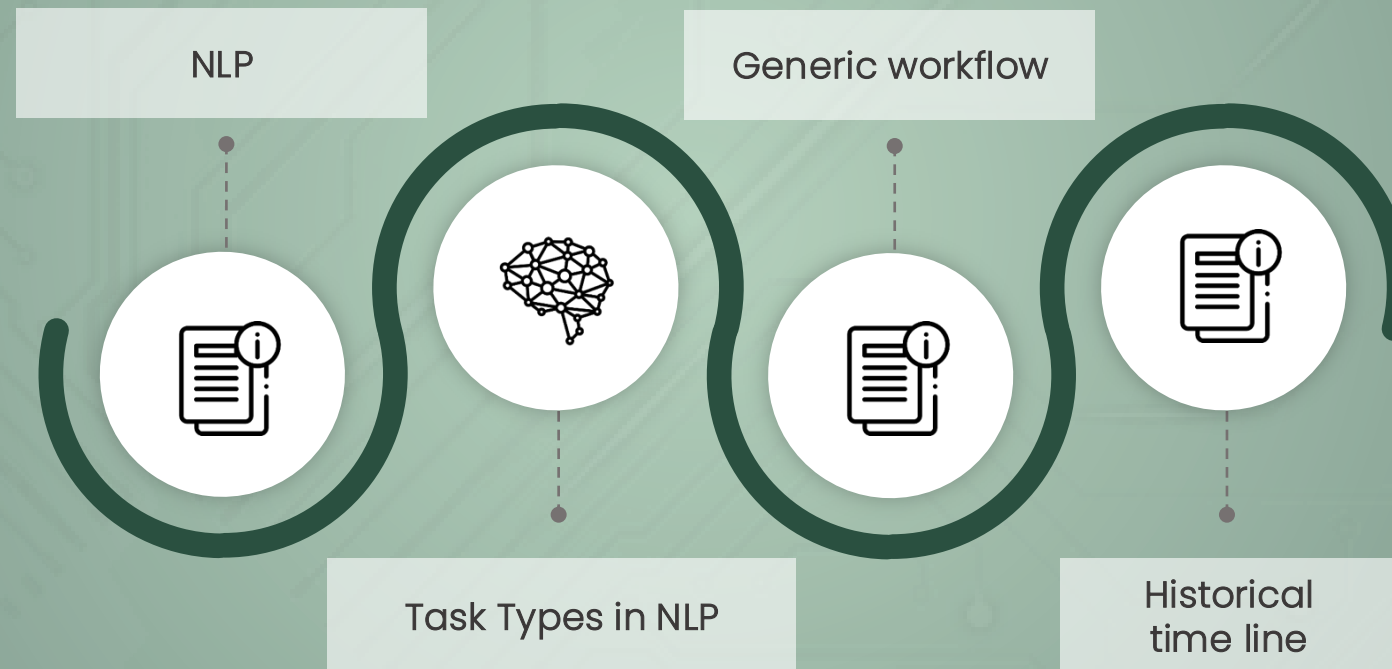
Agenda

- **What is Natural Language Processing?**
- **Text Preprocessing**
 - Tokenization
 - Part-of-Speech-Tagging
 - Stemming & Lemmatization
 - Named Entity Recognition
- **Text Representation Models**
 - Bag-of-Words, N-Grams
 - TF-IDF
 - Word Embeddings
- **Deep Learning**
 - Markov Chain Language Models
 - RNN, LSTM
 - Attention Mechanisms before Transformers



Natural Language Processing

... is a technology that bridges the communication gap between human language and computer understanding.



Natural Language Processing (NLP)

The primary goal of NLP is to enable computers to understand, interpret, and generate human language in a way that is both meaningful and useful.

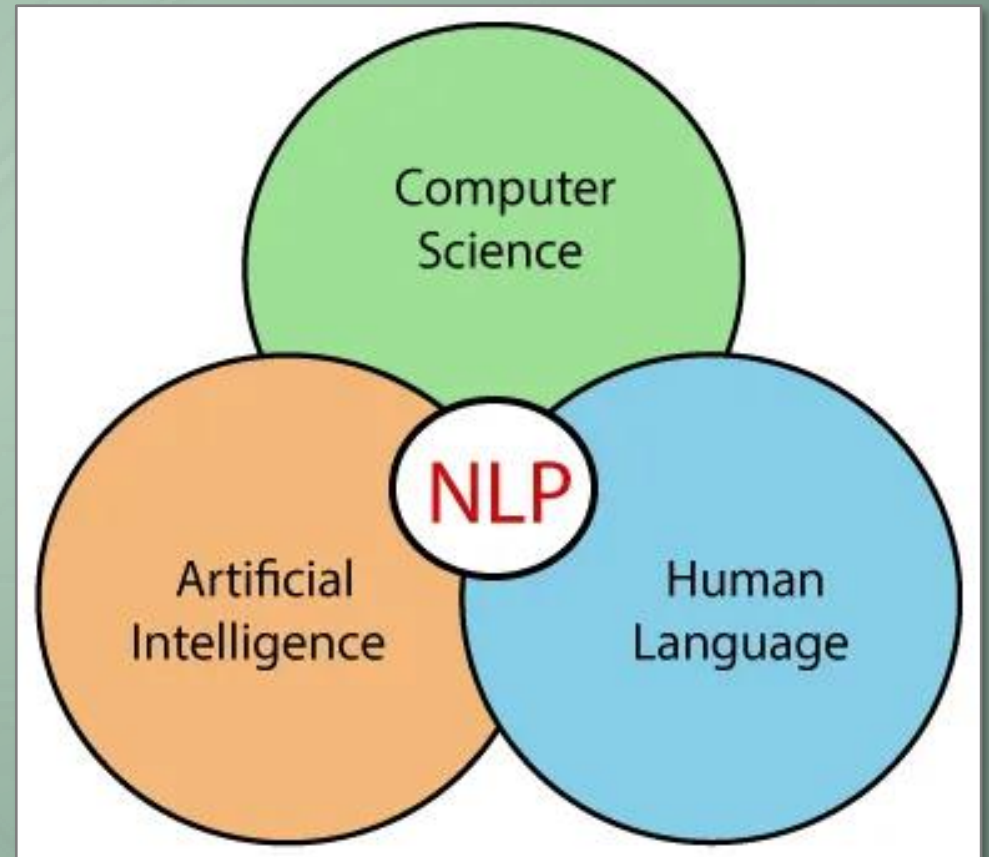
Interdisciplinary Nature

It encompasses areas of

- computer science
- artificial intelligence
- linguistics
- to interpret, recognize, and generate human language in a way that is valuable.

Real-World Applications

- Voice assistants
- Chatbots
- Translation services
- Sentiment analysis
- Customer service.



Task Types in NLP

This table maps key NLP techniques to their applications, showing breadth and depth of natural language processing capabilities across different task types and domains.

Task Type	Techniques	Primary Goal
Preprocessing	▪ Tokenization ▪ Stemming/Lemmatization ▪ POS Tagging, NER	Prepare text for further processing
Classification	▪ Sentiment Analysis ▪ Text Classification ▪ POS Tagging, NER	Assign categories or labels to text
Representation	▪ TF-IDF ▪ N-Grams ▪ Word Embeddings	Convert text to numerical format
Deep Learning	▪ Contextualized Embeddings ▪ RNN/LSTM ▪ Attention	Advanced pattern recognition and understanding
Text Generation	▪ QA Systems ▪ Information Retrieval ▪ Summarization ▪ Programming ▪ Speech-to-Text	Create or produce new text content
Translation	▪ Language Detection ▪ Machine Translation	Convert between languages
Image Generation	▪ Text-to-Image	Create visual content from text

How NLP Works

Generic workflow of Natural Language Processing



Input

Receives text or speech.

Preprocessing

Cleans and converts input. Includes tokenization and stemming.

Context Analysis

Understands structure and meaning. Uses parsing and semantic analysis.

Machine Learning

Applies algorithms for interpretation. Ranges from rule-based to deep learning.

Output

Generates responses or actions. Examples:

- Text generation
- sentiment classification.

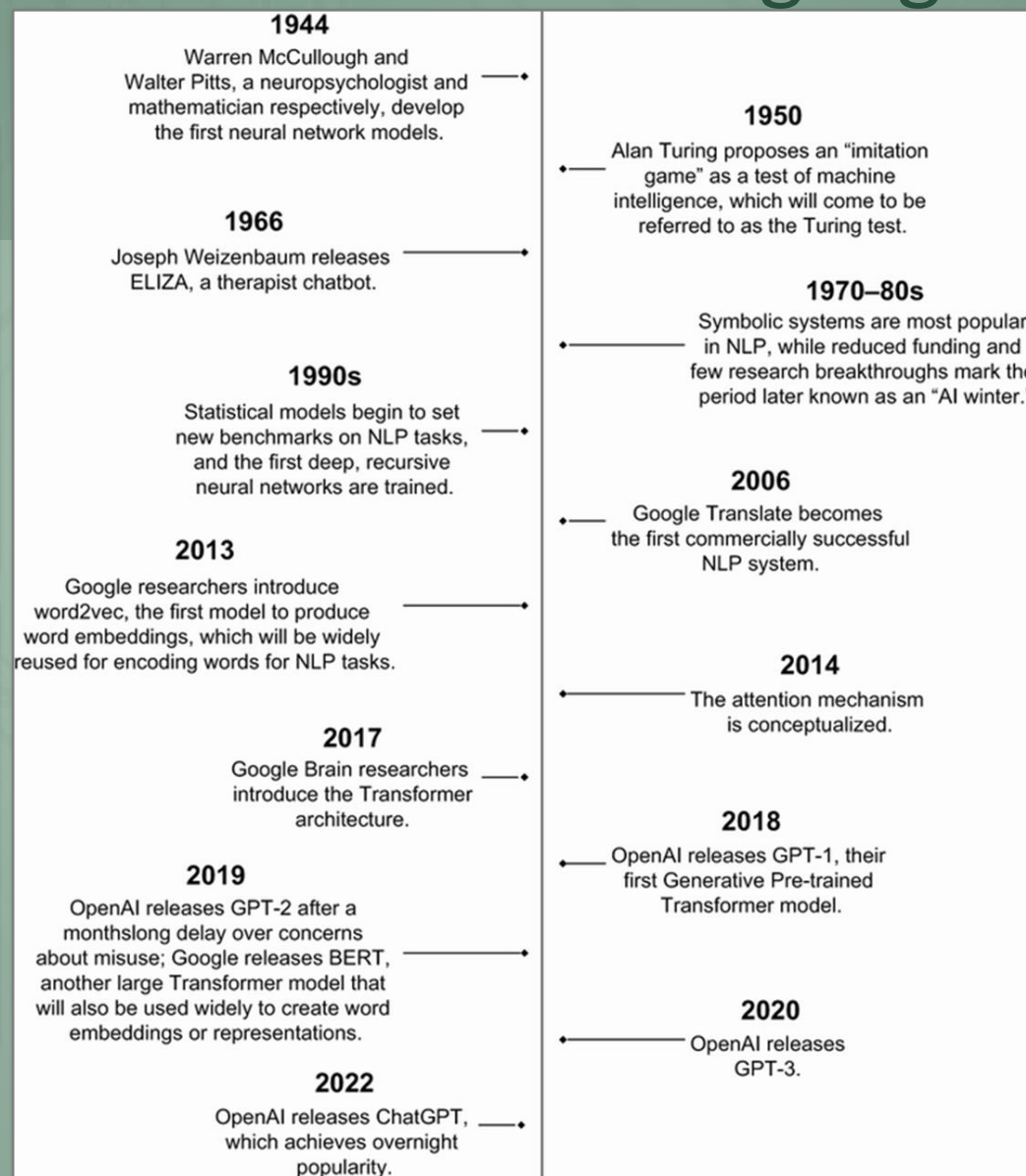
Historical time line of Natural Language Processing

1950s–1960s: Early Days

- Georgetown-IBM experiment, 1954, first machine translation from Russian to English
- Idea by Alan Turing (“Turing test”): When will machines lead conversation in a way that user cannot recognize it as machine?

2010s: Deep Learning Breakthroughs

- Adoption of deep learning and neural networks.
- Emergence of models like Word2Vec (2013) and BERT (2018).



1980–2010: Statistical Revolution

- Statistical models, algorithms like Hidden Markov Models.
- Machine Learning, Language Processing, Word vectorization (Bag-of-Words, TF-IDF, etc.)

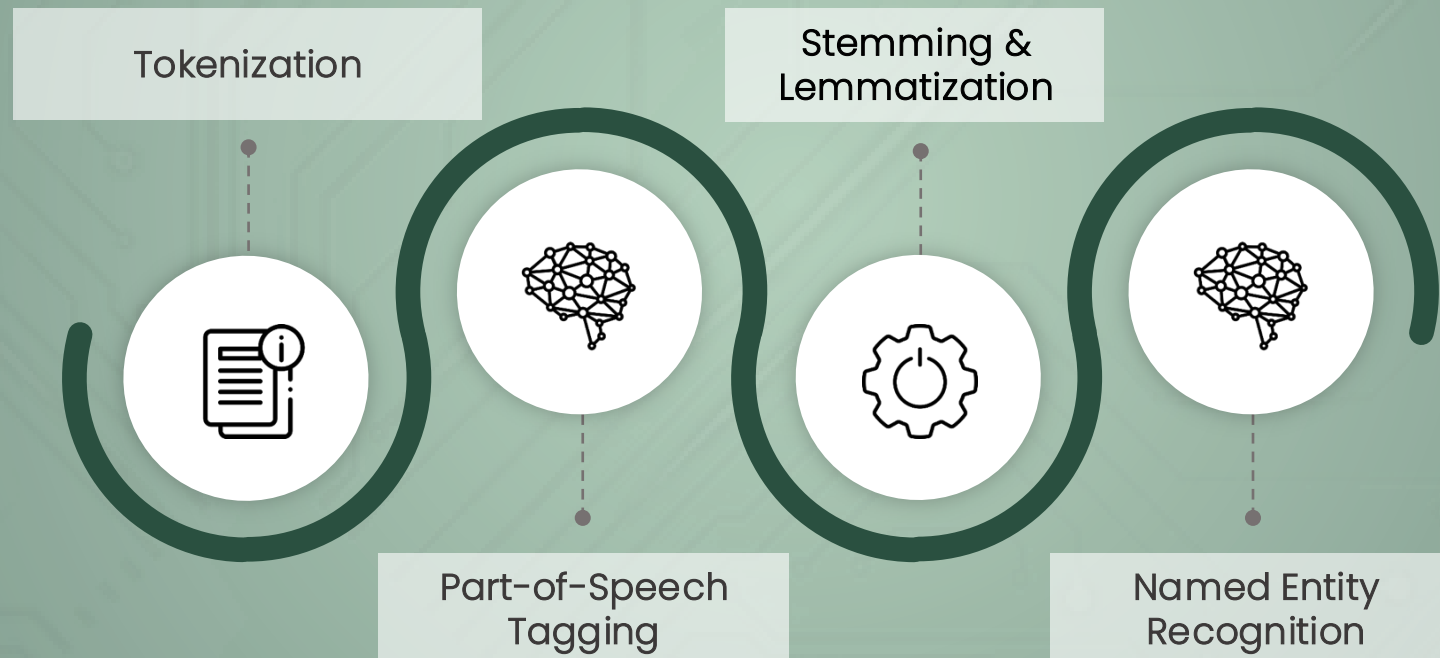
2020s: Advanced Language Models

- State-of-the-art models like GPT and Transformer architectures.
- Unprecedented capabilities in language generation and understanding.

Text Preprocessing: The Foundation of NLP

Text preprocessing forms the essential first stage of any NLP pipeline, transforming raw text into structured format that algorithms can process effectively.

These fundamental techniques establish the groundwork for all subsequent NLP tasks.



Tokenization

... the process of dividing text into smaller units, such as words, characters, or subwords, to facilitate machine processing and analysis

Why is Tokenization Important?

Prepares Text for Machine Processing:

- Tokenization converts unstructured text
- into structured format suitable for machine learning models.

Facilitates Analysis

- By breaking text into manageable parts
- tokenization helps algorithms identify patterns
- and relationships within the data

Supports Multilingual and Complex Texts

- Tokenization can handle diverse languages
- and text structures
- by adapting its granularity
(e.g., word-level or character-level tokens)

"This is the first step in the NLP pipeline"

Tokenizer

'This' 'is' 'the' 'first' 'step' 'in' 'the' 'NLP' 'pipeline'

Tokenization Techniques

... by granularity level

Approach ... Tokenization	Primary Function	Description	Use Cases	Best For
Character ...	Break text into individual characters	Breaks text into individual characters (e.g., "Chatbots" → ["C", "h", "a", "t", "...])	▪ Morphologically rich languages ▪ OCR post-processing ▪ Granular text analysis	Languages without clear word boundaries
Subword ...	Divide words into meaningful subunits	Divides words into smaller meaningful units (e.g., "unusual" → ["un", "usual"])	▪ Out-of-vocabulary handling ▪ Neural machine translation ▪ Multilingual models	▪ Complex morphology ▪ rare words ▪ LLM tasks
Word ...	Split text into individual words	Splits text into individual words (e.g., "The grass is green" → ["The", "grass", "is", "green"])	▪ Search engines ▪ Text parsing ▪ Language modeling	Languages with clear word boundaries (English)
Phrase ...	Group meaningful phrases as tokens	Groups meaningful phrases as single tokens (e.g., "San Diego" → ["San Diego"])	▪ Named entity recognition ▪ Information extraction ▪ Domain-specific parsing	Entity recognition tasks
Sentence ...	Split text into sentences	Splits paragraphs into sentences for analysis	▪ Text summarization ▪ Sentiment analysis ▪ Document structure analysis	Document-level processing

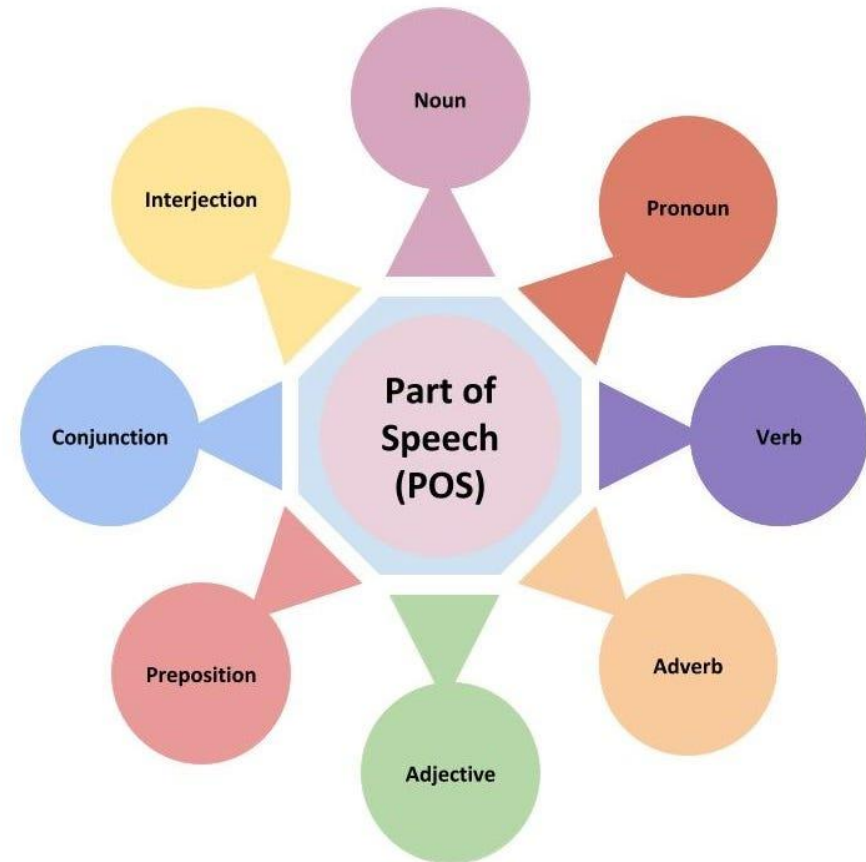
Part-of-Speech (POS) Tagging

... assigns grammatical categories (e.g., noun, verb, adjective) to words in a sentence by analyzing their context, structure, and predefined linguistic rules

How It Works

- Tokenization:
Splits text into words for analysis.
- Context Analysis:
Resolves ambiguities based on surrounding words.
- Uses algorithms like:
 - Hidden Markov Models
 - Rule-based systems
 - Nowadays: Transformer based approaches (BERT)

article verb article
The cat sat on the mat
noun preposition noun



Stemming: Simplifying Words for NLP

Stemming reduces words to their root or base form by removing prefixes and suffixes (e.g., "running," "runs," "ran" → "run")

Purpose

- Helps group related words for efficient text analysis and information retrieval.

Advantages

- Reduces dimensionality of text data.
- Improves search relevance and document classification.
- Speeds up NLP tasks with simplified word forms.

Challenges

- May produce non-meaningful stems ("argue" → "argu").
- Over-stemming and under-stemming can affect accuracy

Example:

- "Fishing," "fished," "fisher" → "fish."

Original Word	Stemmed Word	Explanation
Running	Run	Removes suffix "-ing"
Studies	Studi	Removes plural suffix "-es"
Happily	Happi	Removes suffix "-ly"
Fishing	Fish	Removes suffix "-ing"
Argument	Argu	Removes suffix "-ment"
National	Nation	Removes suffix "-al"
Organization	Organ	Removes suffix "-ization"

Types of Stemming in NLTK (Python Package)

NLTK provides several stemming algorithms, each with unique characteristics and use cases. Here are the main types:

Stemmer	Description	Example	Typical Use Cases
Porter Stemmer	A widely-used algorithm that applies heuristic rules to remove suffixes.	"running" → "run"; "flies" → "fli"	Search engines and text classification.
Snowball Stemmer	An improved version of Porter Stemmer, supporting multiple languages.	"generously" → "generous"	Multilingual NLP tasks and sentiment analysis.
Lancaster Stemmer	A faster and more aggressive algorithm, often resulting in shorter stems.	"running" → "run"; "organization" → "organ"	Applications requiring high-speed preprocessing.
Regex Stemmer	Uses regular expressions to define stemming rules for specific suffixes.	"running" → "runn"; "workable" → "work"	Custom text preprocessing for domain-specific tasks.
ARLSTem Stemmer	Designed for Arabic language stemming, focusing on word morphology.	) 'عمل' → 'يعمل'(Arabic)	Arabic NLP applications like search and translation.
RSLP Stemmer	A stemmer tailored for Portuguese language processing.	Example specific to Portuguese	Portuguese text analysis and sentiment detection.

Lemmatization: Context-Aware Text Normalization

Lemmatization reduces words to their dictionary form (lemma) by analyzing context, meaning, and part of speech (e.g., "running," "ran," "runs" → "run")

How Lemmatization Works

- **Tokenization:**
Breaks text into words or phrases for processing.
- **POS Tagging:**
Assigns grammatical categories (e.g., noun, verb) to each word.
- **Morphological Analysis:**
Identifies roots, prefixes, and suffixes of words.
- **Context Analysis:**
Considers surrounding words to determine meaning and usage.
- **Dictionary Lookup:**
Matches words to their base forms in a lexicon.
- **Output Generation:**
Replaces words with their lemmas for accurate representation.

Original Word	Lemmatized Word	Part of Speech	Explanation
Running	Run	Verb	Converts to base form of the verb.
Studies	Study	Noun	Removes plural suffix "-es".
Happily	Happy	Adjective	Converts adverb to its base adjective.
Better	Good	Adjective	Context-based transformation.
Was	Be	Verb	Converts past tense to base form.
Children	Child	Noun	Converts plural noun to singular form.
Leaves	Leaf	Noun	Removes plural suffix "-es".
Singing	Sing	Verb	Converts verb to its root form.
Geese	Goose	Noun	Converts irregular plural to singular.

Stemming vs. Lemmatization

Stemming reduces words to their root form by removing affixes without considering context. Lemmatization transforms words into dictionary base form (lemma) using context and part-of-speech analysis.

Aspect	Stemming	Lemmatization
Definition	- Reduces words to their root form - by chopping off suffixes or prefixes	- Reduces words to their dictionary base form (lemma) - considering context and POS
Method	- Rule-based approach - often using heuristics	- Uses vocabulary - morphological analysis - part-of-speech tagging
Output	May produce non-meaningful stems (e.g., "studies" → "studi")	Always produces valid words (e.g., "studies" → "study")
Context Awareness	- Ignores context - operates on individual words	- Context-aware - considers sentence structure and word usage
Speed	Faster due to simplicity	Slower due to complex analysis
Accuracy	- Less accurate - may over-stem or under-stem words	- More accurate - ensures meaningful base forms
Examples	"Caring" → "Car," "Studies" → "Studi"	"Caring" → "Care," "Studies" → "Study"
Use Cases	- Information retrieval - search engines - quick preprocessing tasks	- Machine translation - sentiment analysis - tasks requiring linguistic accuracy

Named Entity Recognition (NER)

... is an advanced NLP technique that identifies and classifies named entities in text into predefined categories such as people, organizations, locations, dates, and more

Example:

Apple CEO **Tim Cook** announced a new partnership with **Microsoft** on **Thursday** in **New York**.

Tesla reported record profits of **\$3.3 billion** for **Q1 2023**, according to **Elon Musk**.

The **World Health Organization** issued new guidelines for countries in **Europe** **yesterday**.

Dr. Sarah Johnson from **Stanford University** published a research paper about **climate change** in **Nature**.

Common Entity Types

- PERSON: Names of people
- ORG: Organizations, companies
- LOC: Locations, countries, cities
- DATE: Dates and times
- GPE: Geopolitical entities
- MONEY: Monetary values

Aspects

- Customized entity types are also possible.
- Advanced tools like BERT (Transformer based) are used.

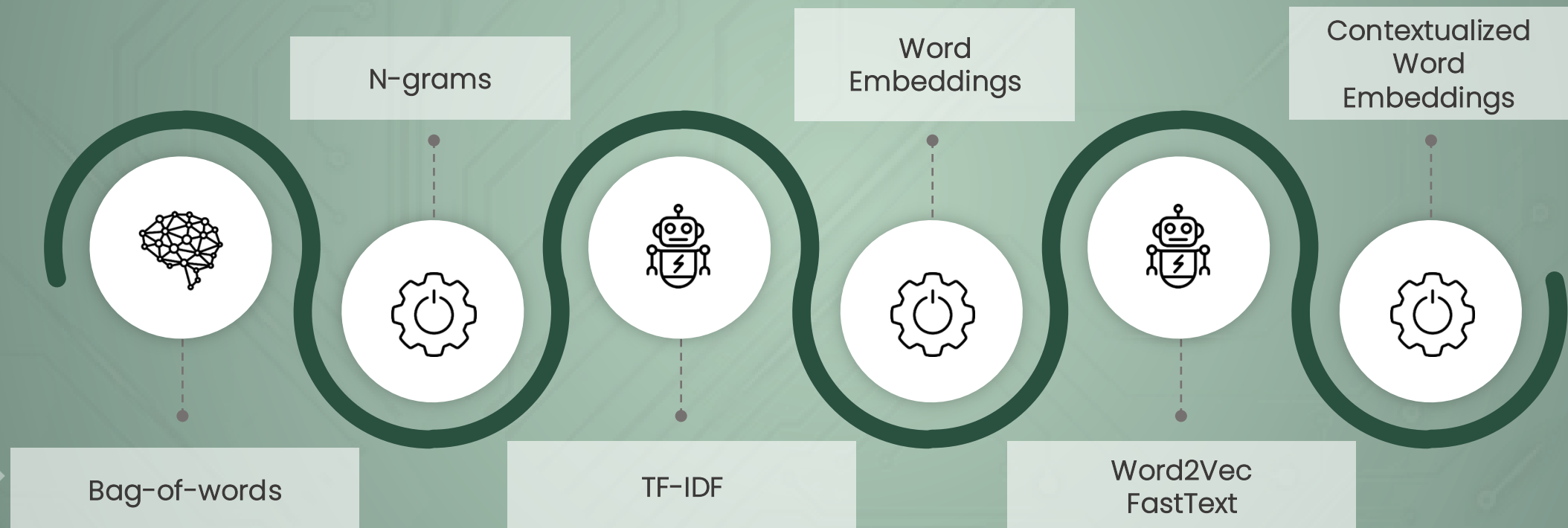
Use Case Overview: POS, Stemming/Lemmatization, NER

Grammatical and Semantic Analysis Techniques for Text Understanding

Approach	Primary Function	Use Cases	Best For
POS Tagging (HMM)	Assign grammatical categories	- Grammar checking - Information extraction - Syntactic parsing	Traditional NLP pipelines
POS Tagging (BERT)	Context-aware grammatical tagging	- Advanced grammar checking - Semantic role labeling - Question answering	Modern NLP applications
Porter Stemmer	Reduce words to root forms	- Search engines - Text classification - Information retrieval	English text processing
Lemmatization	Context-aware word normalization	- Machine translation - Sentiment analysis - Knowledge graphs	High-accuracy applications
Named Entity Recognition	Identify entities in text	- Chatbots - Financial analysis - Healthcare systems - Legal document processing	Domain-specific applications

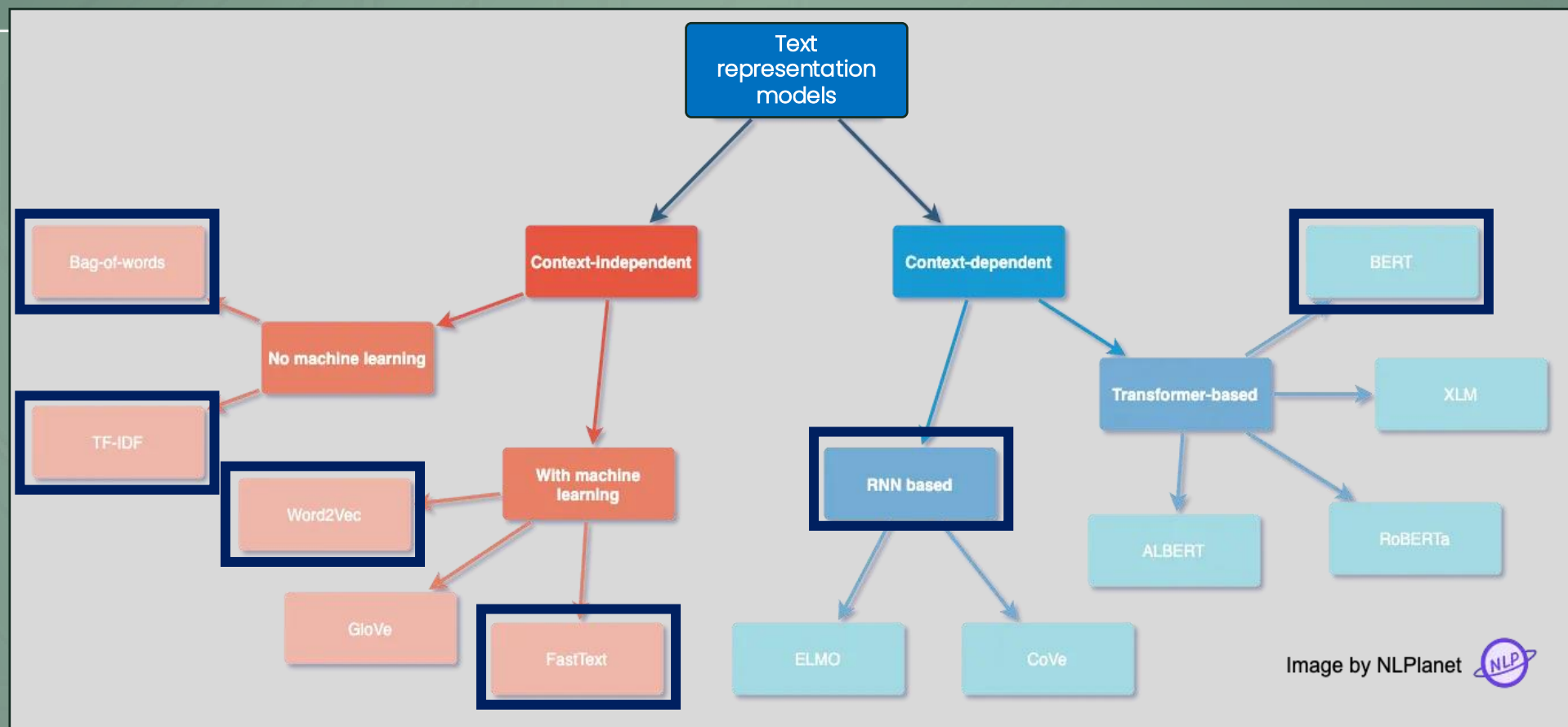
Text Representation Models

... convert text data into numerical formats, such as vectors or embeddings, to enable machine learning algorithms to process and analyze language by capturing its structure, frequency, or semantic meaning



Overview of text representation models

... division between context-independent and context-dependent methods including transformer-based architectures



Bag-of-Words representation

... transforms text into numerical vector, where each unique word is represented by feature and value indicates frequency of word in the document, disregarding grammar, word order.

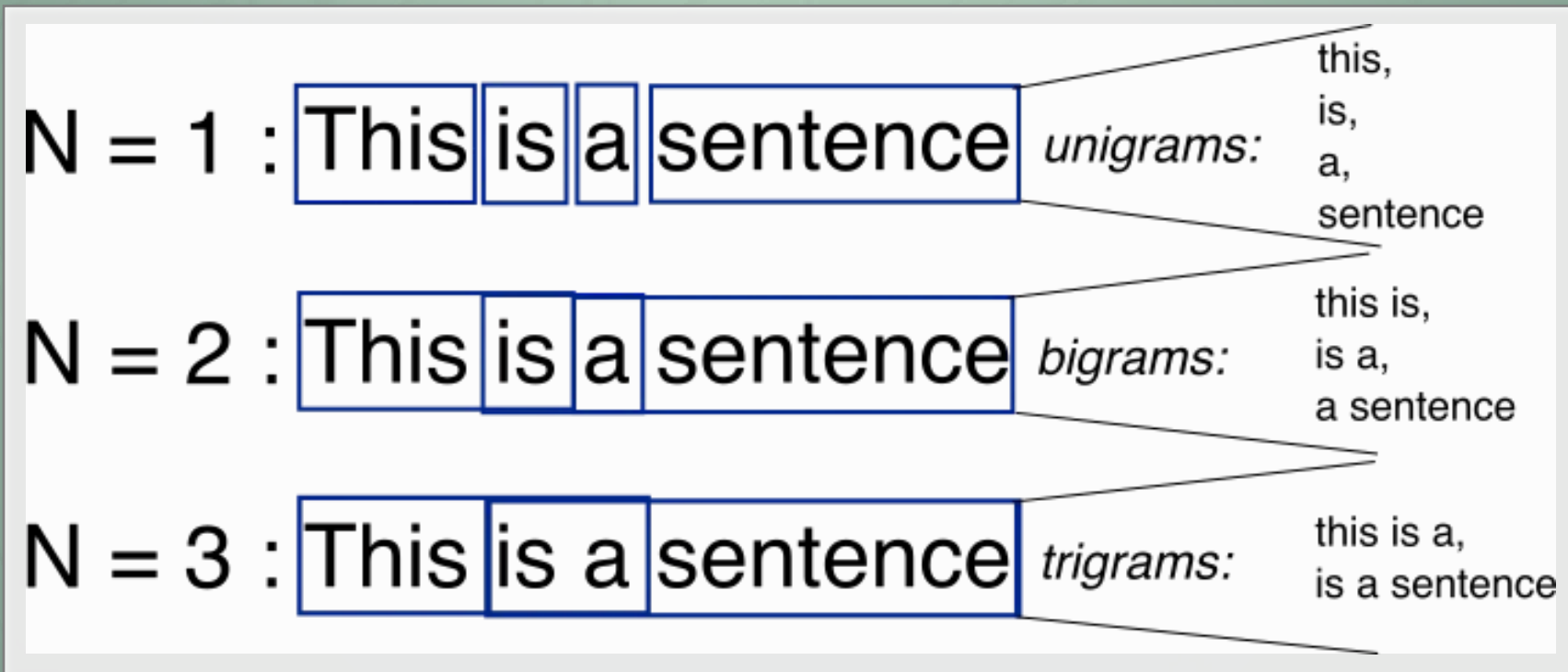
	about	bird	heard	is	the	word	you
About the bird, the bird, bird bird bird	1	5	0	0	2	0	0
You heard about the bird	1	1	1	0	1	0	1
The bird is the word	0	1	0	1	2	1	0

Limitations of Bag of Words

- Ignores word order
- Fails to capture phrases and context.

N-Gram: Extending the Bag of Words Model

N-grams are contiguous sequences of n items from a given text or speech.
Extends BoW by counting occurrences of n -grams instead of individual words.



Typical application: Autocompletion

Term Frequency-Inverse Document Frequency (TF-IDF)

... measures how important a term is within a specific document relative to a collection of documents (corpus)

Components

- **Term Frequency (TF):**
Measures how often a word appears in a document.
- **Inverse Document Frequency (IDF):**
Evaluates how rare or unique a word is across all documents

TF - IDF

$$\text{TF-IDF} = \text{TF}(t,d) \times \text{IDF}(t)$$

↑ ↑
Term frequency Inverse document frequency



Idea:

- If word occurs often in document A
- and rare in all other documents
- then it is connected with document A

TF-IDF: Numeric Example

Document 1: "The cat sat on the mat"

Document 2: "The dog chased the cat"

Document 3: "The cat and the dog played"

$TF(t,d) = (\text{Number of times term } t \text{ appears in document } d) / (\text{Total number of terms in document } d)$

Document 1:

- $TF("the") = 2/6 = 0.333$
- $TF("cat") = 1/6 = 0.167$
- $TF("sat") = 1/6 = 0.167$
- $TF("on") = 1/6 = 0.167$
- $TF("mat") = 1/6 = 0.167$

Document 2:

- $TF("the") = 2/5 = 0.400$
- $TF("dog") = 1/5 = 0.200$
- $TF("chased") = 1/5 = 0.200$
- $TF("cat") = 1/5 = 0.200$

Document 3:

- $TF("the") = 2/6 = 0.333$
- $TF("cat") = 1/6 = 0.167$
- $TF("and") = 1/6 = 0.167$
- $TF("dog") = 1/6 = 0.167$
- $TF("played") = 1/6 = 0.167$

$IDF(t) = \log(\text{Total number of documents} / \text{Number of documents containing term } t)$

- $IDF("the") = \log(3/3) = \log(1) = 0$
- $IDF("cat") = \log(3/3) = \log(1) = 0$
- $IDF("dog") = \log(3/2) = \log(1.5) \approx 0.176$
- $IDF("sat") = \log(3/1) = \log(3) \approx 1.099$
- $IDF("on") = \log(3/1) = \log(3) \approx 1.099$
- $IDF("mat") = \log(3/1) = \log(3) \approx 1.099$
- $IDF("chased") = \log(3/1) = \log(3) \approx 1.099$
- $IDF("and") = \log(3/1) = \log(3) \approx 1.099$
- $IDF("played") = \log(3/1) = \log(3) \approx 1.099$

TF-IDF: Numeric Example

Document 1: "The cat sat on the mat"

Document 2: "The dog chased the cat"

Document 3: "The cat and the dog played"

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

Document 1:

- $\text{TF-IDF}(\text{"the"}) = 0.333 \times 0 = 0$
- $\text{TF-IDF}(\text{"cat"}) = 0.167 \times 0 = 0$
- $\text{TF-IDF}(\text{"sat"}) = 0.167 \times 1.099 \approx 0.184$
- $\text{TF-IDF}(\text{"on"}) = 0.167 \times 1.099 \approx 0.184$
- $\text{TF-IDF}(\text{"mat"}) = 0.167 \times 1.099 \approx 0.184$

Document 2:

- $\text{TF-IDF}(\text{"the"}) = 0.400 \times 0 = 0$
- $\text{TF-IDF}(\text{"dog"}) = 0.200 \times 0.176 \approx 0.035$
- $\text{TF-IDF}(\text{"chased"}) = 0.200 \times 1.099 \approx 0.220$
- $\text{TF-IDF}(\text{"cat"}) = 0.200 \times 0 = 0$

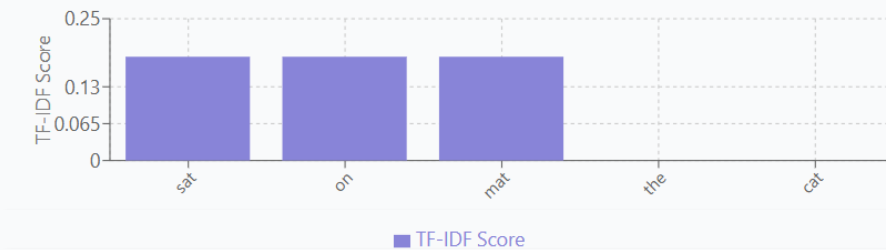
Document 3:

- $\text{TF-IDF}(\text{"the"}) = 0.333 \times 0 = 0$
- $\text{TF-IDF}(\text{"cat"}) = 0.167 \times 0 = 0$
- $\text{TF-IDF}(\text{"and"}) = 0.167 \times 1.099 \approx 0.184$
- $\text{TF-IDF}(\text{"dog"}) = 0.167 \times 0.176 \approx 0.029$
- $\text{TF-IDF}(\text{"played"}) = 0.167 \times 1.099 \approx 0.184$

TF-IDF: highlights most distinctive terms for each document

- Terms that appear in all documents ("the", "cat") have $TF\text{-}IDF = 0$
- Unique terms have the highest TF-IDF scores in each document
- "dog" has a lower but non-zero score as it appears in two documents

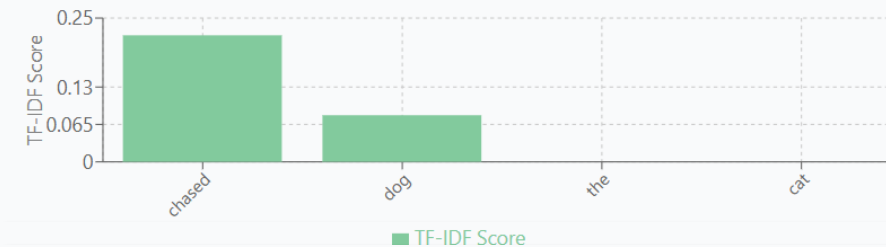
Document 1: "The cat sat on the mat"



Key Insight:

- Unique terms 'sat', 'on', and 'mat' have the highest scores
- while common terms have zero value.

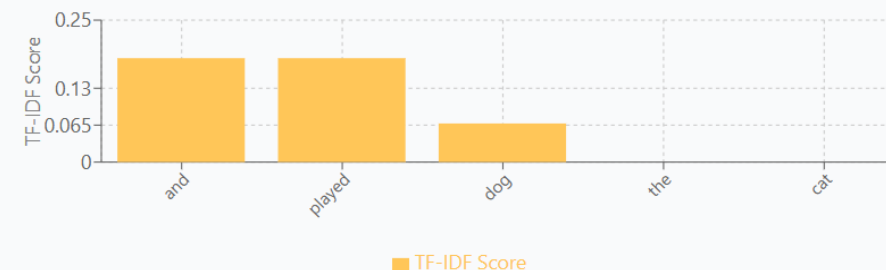
Document 2: "The dog chased the cat"



Key Insight:

- Unique term 'chased' has highest score
- followed by 'dog' which appears in two documents

Document 3: "The cat and the dog played"



Key Insight:

- Unique terms 'played' and 'and' have highest scores,
- showing their importance to this document

Word Embeddings: Transforming Words into Vectors

They enable us to convert textual data into numerical forms that can be processed by machine learning models.

Educational starting point: One-Hot Encoding

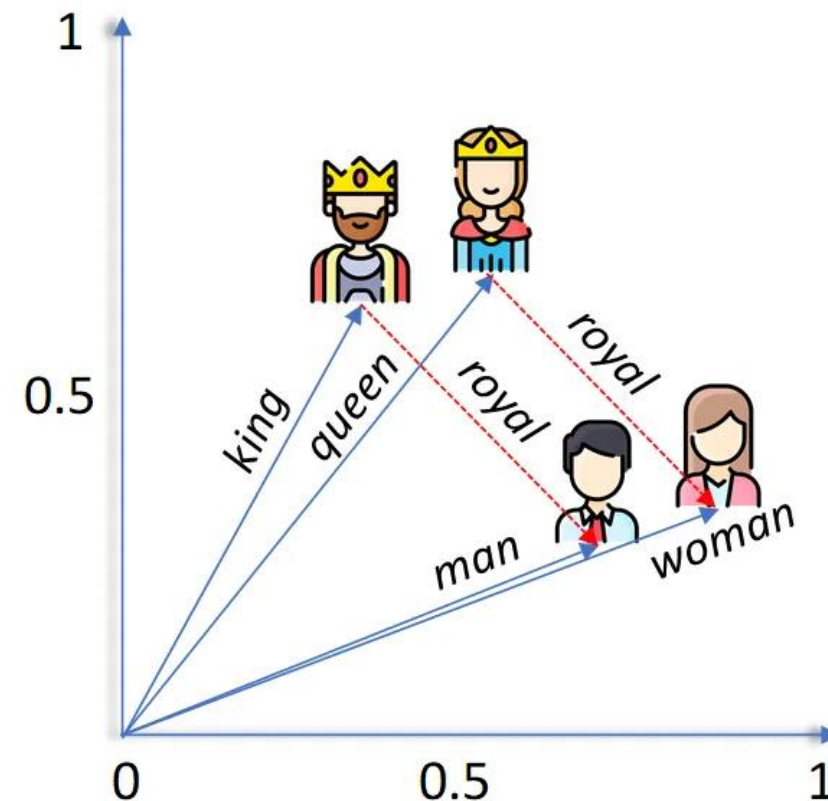
- process that converts categorical variables into binary vector representation
- only one element is "hot" (set to 1)
- all other elements are "cold" (set to 0), uniquely representing each category.

Semantic similarity: Word2Vec Models

- words are represented as vectors in continuous vector space

Contextualized Word Embedding

- meaning of a word is dynamically encoded
- based on the word's context within a sentence
- leading to different embeddings for the same word in different contexts
- thereby capturing nuances in usage and meaning

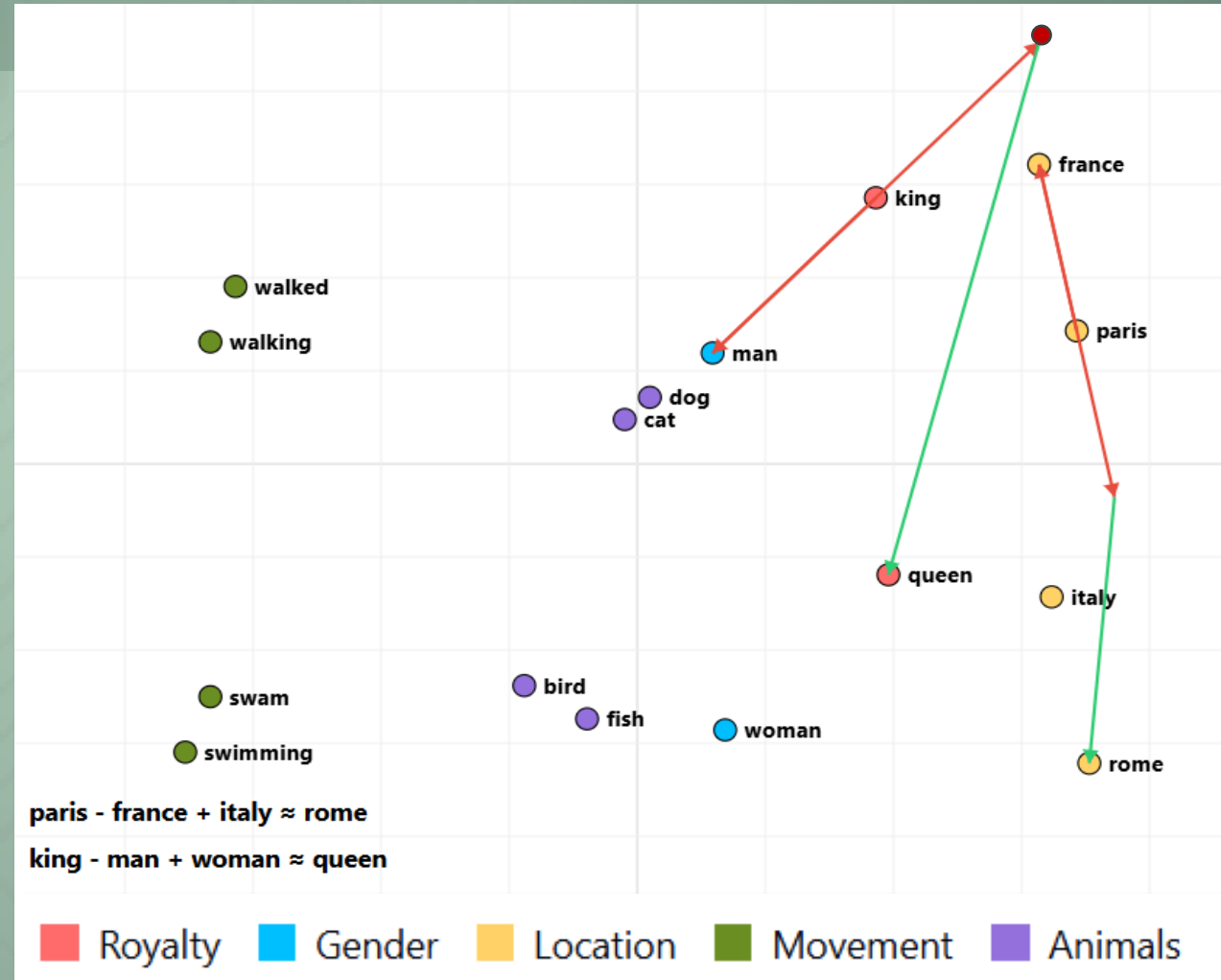


Transforming Words into Vectors

Textual data is transformed into numerical forms that can be processed by ML models.

Vector Space Properties

- Words with similar meanings cluster together in vector space
- Vector direction often represents specific semantic relationships
- Vector arithmetic reveals analogies and conceptual relationships
- High-dimensional spaces (typically 100-300 dimensions) capture nuanced semantic information



Cosine similarity

... is metric used to measure how similar two documents represented by vectors are irrespective of their size. It is commonly used for semantic search in vector databases.

It is measured by the cosine of the angle between two vectors and determines whether two vectors are pointing in roughly the same direction.

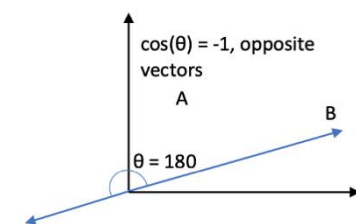
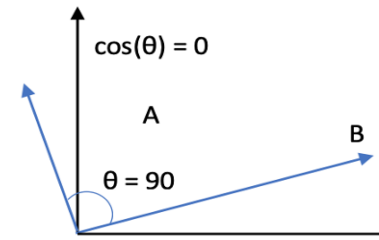
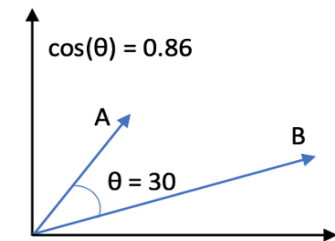
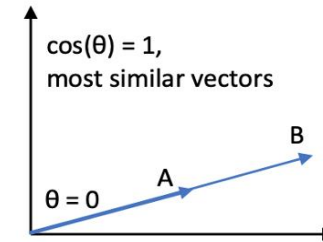
Formula

$$\cos \theta = \frac{\vec{a} \times \vec{b}}{\|\vec{a}\| \times \|\vec{b}\|} \text{ with } \|\vec{a}\| = \sqrt{a_1^2 + a_2^2 + \dots + a_n^2} \text{ and } \|\vec{b}\| = \sqrt{b_1^2 + b_2^2 + \dots + b_n^2}$$

- $\vec{a}$ and $\vec{b}$ are vectors, $\vec{a} \times \vec{b}$ is their dot product
- $\|\vec{a}\|$ and $\|\vec{b}\|$: magnitudes i.e. lengths of vectors $\vec{a}$ and $\vec{b}$

Explanation

- Ranges from -1 meaning exactly opposite, to 1 meaning exactly the same
- 0 usually indicating independence
- in-between values indicating intermediate similarity or dissimilarity



One-hot encoding

... is basic method for vectorizing words in NLP. Used as educational starting point

Each word in a vocabulary is represented as a binary vector

- A vector of all zeros except for a single 1
- indicating the word's presence.

Simple and intuitive, but has limitations

- Doesn't capture semantic relationships between words.
- High dimensionality in large vocabularies.

Cosine similarities are always 0 between any two different words

- No semantic relationships captured
- All words equally distant
- No gradual similarity scale

→ This is why word embeddings like Word2Vec were developed

	1	0	0
Index:	0	1	2
	0	1	0
Index:	0	1	2
	0	0	1
Index:	0	1	2

Word2Vec

... is family of shallow neural network-based technique that transforms words into continuous vector representations by capturing their semantic relationships from large text corpora

Key Features

- Captures semantic meaning
- Words with similar meanings are closer in the vector space

Two Training Methods

- Skip-Gram: Predicts neighbor words from a given target word
- Continuous Bag of Words (CBOW): Predicts a word given its context

Benefits

- Enables better performance in NLP tasks
- Helps in capturing semantic relationships (e.g., "king" - "man" + "woman" $\approx$ "queen")

Word2Vec



Continuous bag of words: "I love drinking apple smoothies"

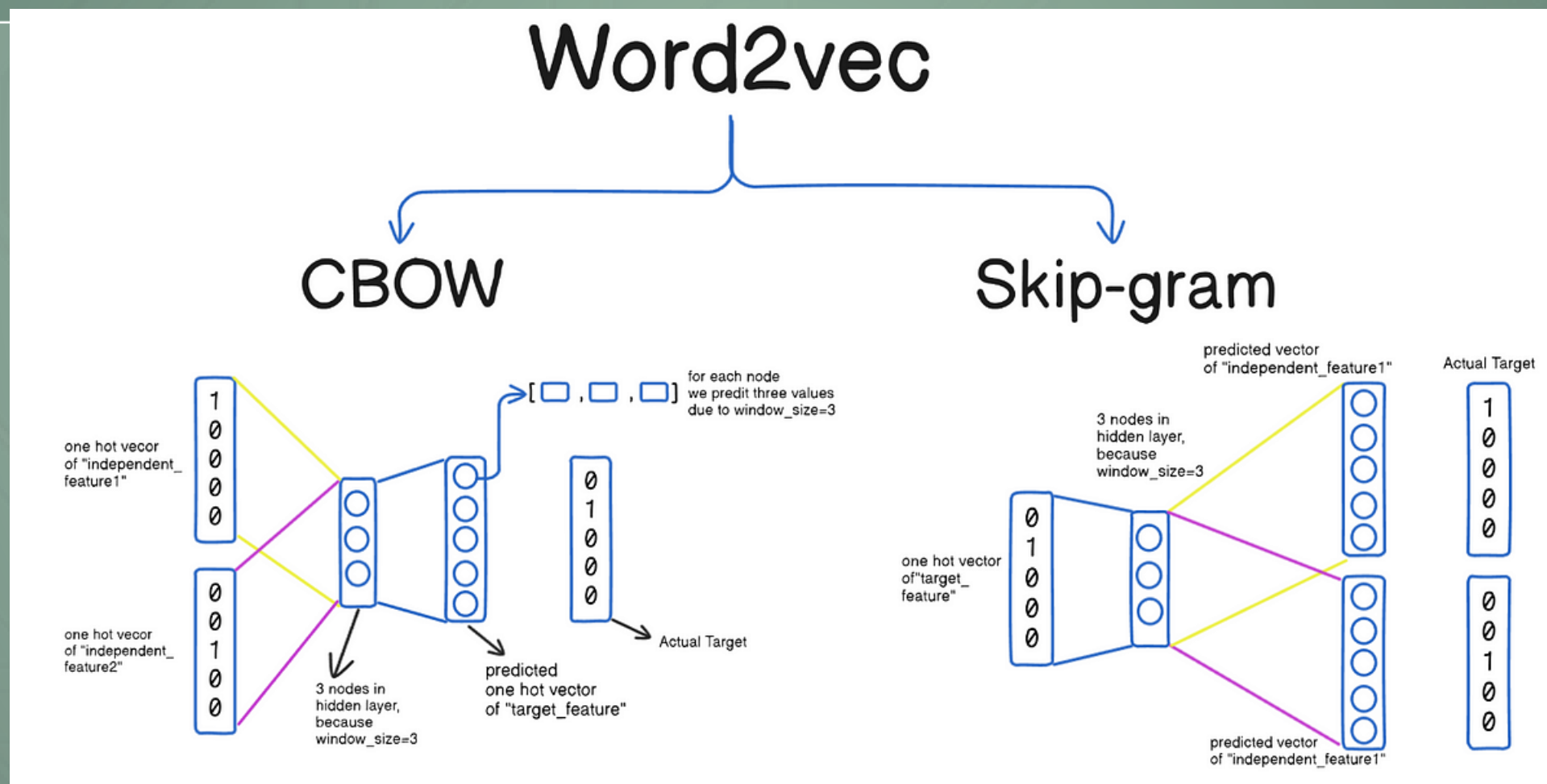


Skip-gram: "I love drinking apple smoothies"

Comparing Word2Vec Training Architectures

CBOW: Uses context words to predict the target word

Skip-Gram: Uses target word to predict context words



Word2Vec: Continuous Bag-of-Words (CBOW)

... is neural network model, predicts target word based on its context words. It's used to generate word embeddings, i.e. vector representations of words that capture semantic relationships.

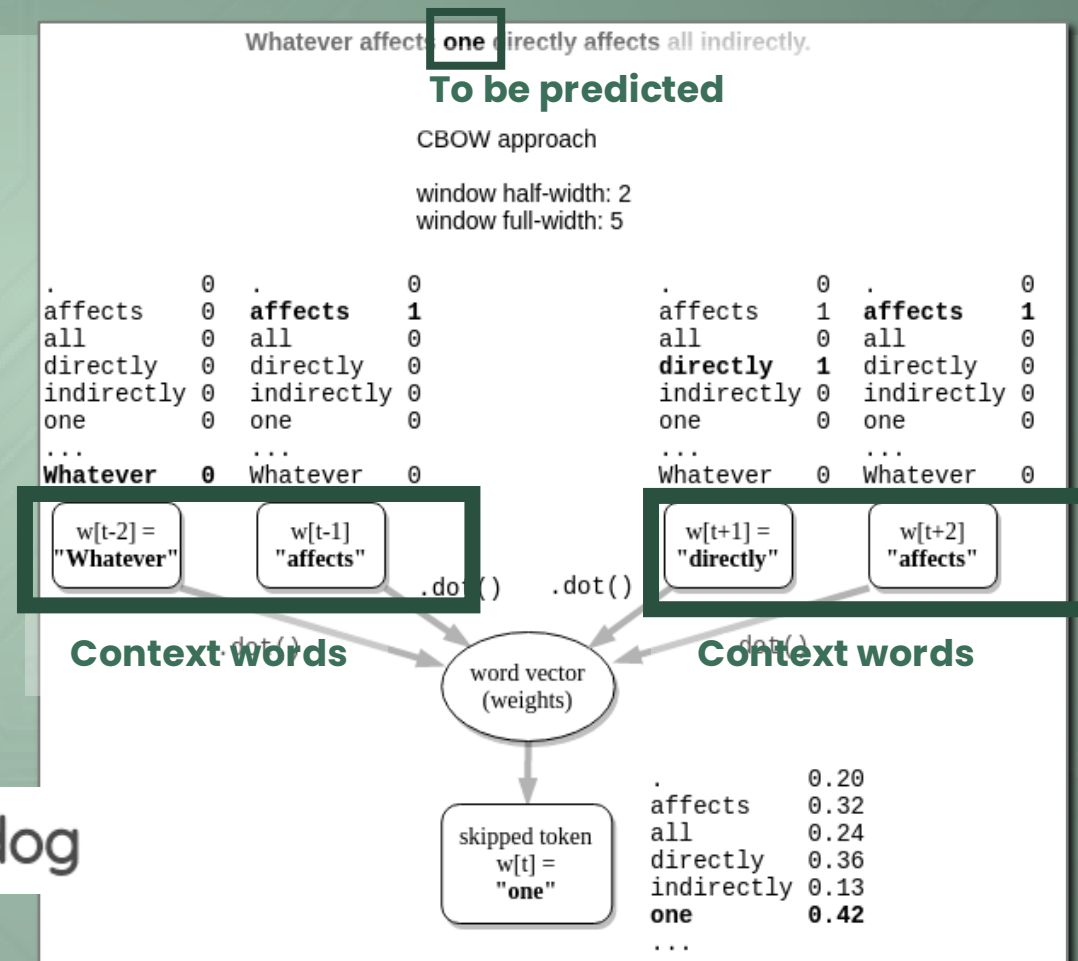
Key Features

- Predicts target word given its direct vicinity words
- Input: Average of vectors of direct vicinity words
- Output: Probability distribution over all words in vocabulary

Advantages and disadvantages

- Performs well on frequent words
- Useful for tasks prioritizing overall context over word order
- However, struggles with out-of-vocabulary words

A quick brown fox jumps over the lazy dog



Word2Vec: Skip-Gram

Predicting Context / Neighbor Words from a Target Word

Core Idea

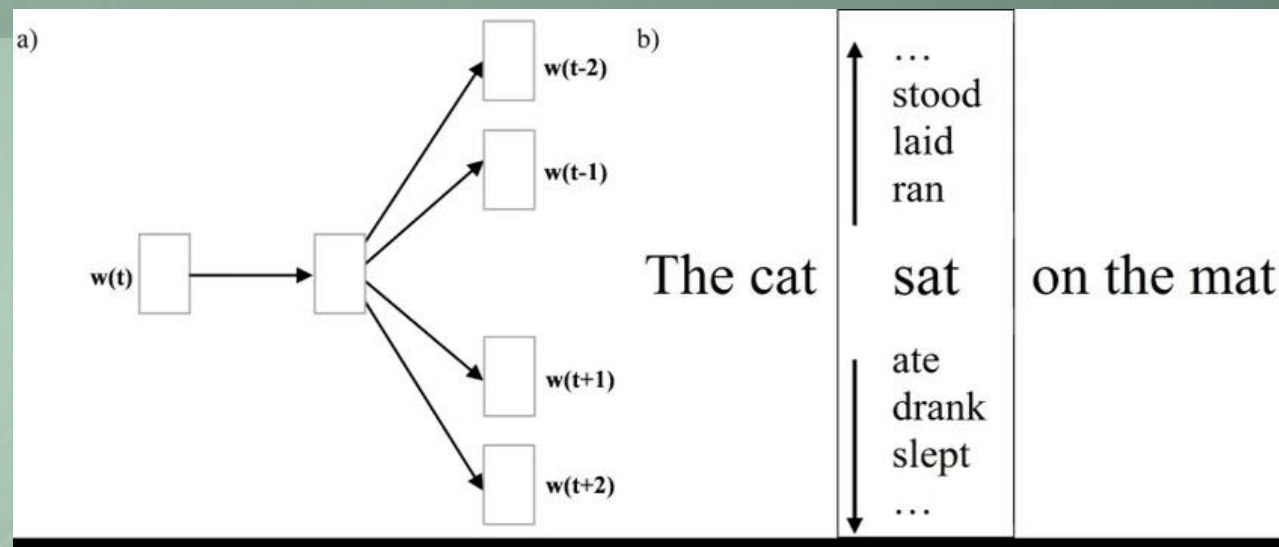
- Learns word embeddings by predicting surrounding context words given a target word within defined window.
- Captures both semantic, syntactic relationships between words, placing similar words close together in vector space.

Model Architecture

- Utilizes a simple neural network with one hidden layer: Input Layer (target word) → Hidden Layer (embedding) → Output Layer (context words).
- Trained on large text corpora using techniques like negative sampling for efficiency.

Strengths and Use Cases

- Excels at representing rare or low-frequency words and capturing rich word relationships.
- Widely used in NLP tasks such as sentiment analysis, machine translation, and semantic similarity calculations



A quick brown fox jumps over the lazy dog

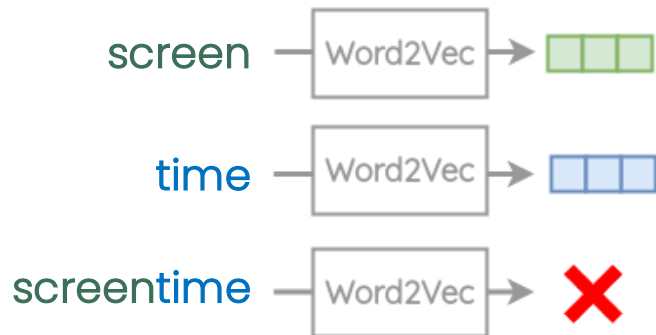
Word2Vec: Limitations

Key Challenges: Out-of-Vocabulary Words and Morphological Blindness

- **Out of Vocabulary(OOV) Words:**

In Word2Vec, an embedding is created for each word. As such, it can't handle any words it has not encountered during its training.

For example, words such as "tensor" and "flow" are present in the vocabulary of Word2Vec. But if you try to get embedding for the compound word "tensorflow", you will get an **out of vocabulary error**.



- **Morphology:**

For words with same radicals such as "eat" and "eaten", Word2Vec doesn't do any parameter sharing. Each word is learned uniquely based on the context it appears in. Thus, there is scope for utilizing the internal structure of the word to make the process more efficient.

Shared radical

eat eats eaten eater eating

Word2Vec extension: Fasttext

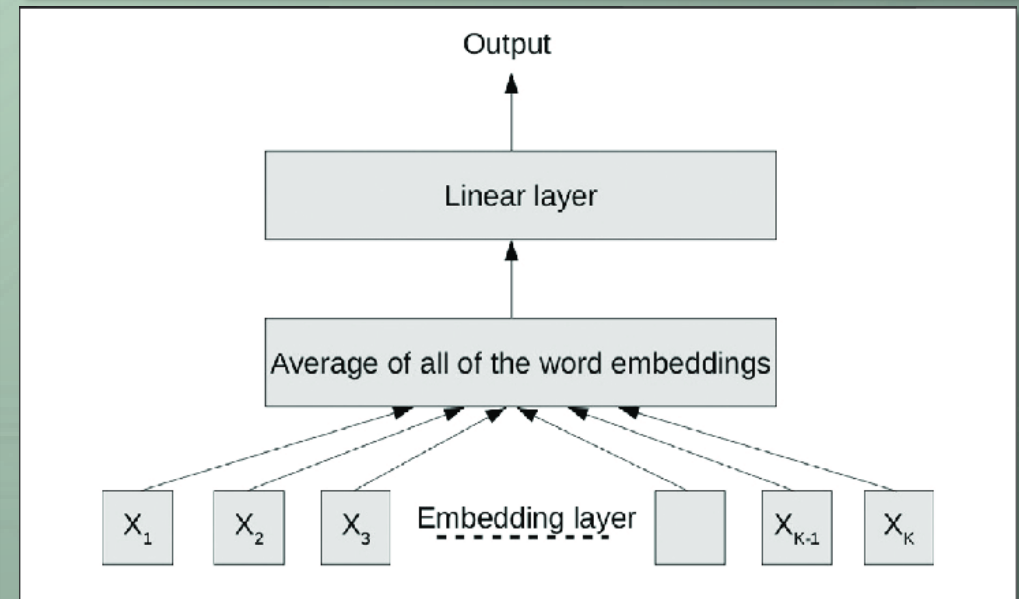
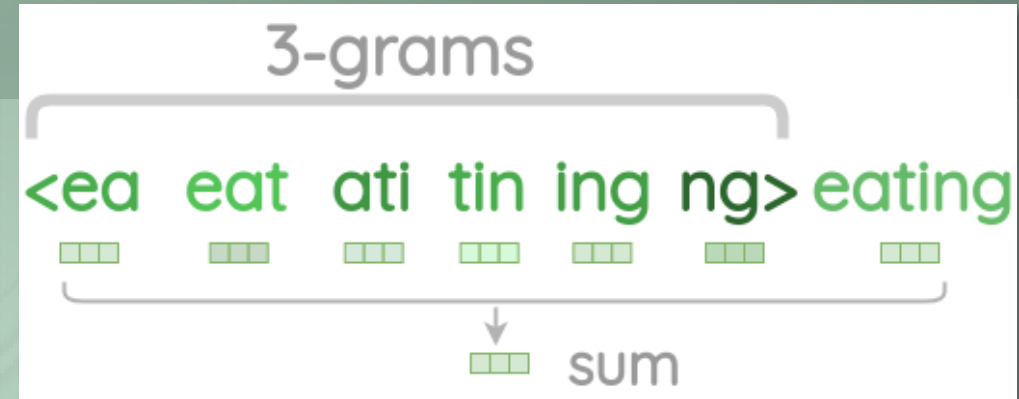
... is subword-based word embedding that processes subword units to capture morphological patterns and word structure

- Represents words as bag of character n-grams (e.g., 3 to 6 characters)
- Each word vector is sum or average of its n-gram vectors plus whole word vector
- Enables embedding of rare or unseen words by composing from known subwords

How FastText Works

- Breaks each word into overlapping character n-grams with boundary symbols (e.g., < and >)
- Learns vector representations for each n-gram during training
- Combines n-gram vectors to form final word embedding, allowing generalization to unseen words

→ Improves handling of morphologically rich languages and noisy text data



Contextualized Word Embedding

Unlike traditional embeddings, contextualized embeddings generate word representations based on their specific context within a sentence, allowing for dynamic meanings.

Key Features

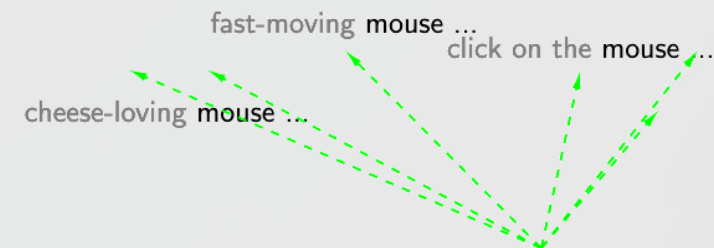
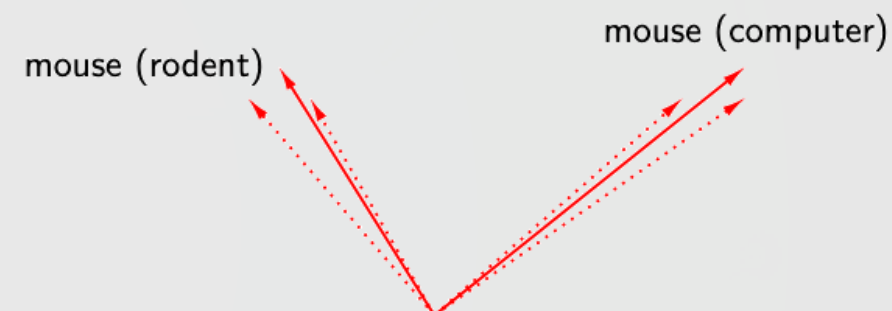
- Words can have different vectors in different contexts.
- Captures polysemy: a word's ability to have multiple meanings.

Benefits

- Enhanced understanding of word nuances and meanings.
- Improved performance on downstream NLP tasks.

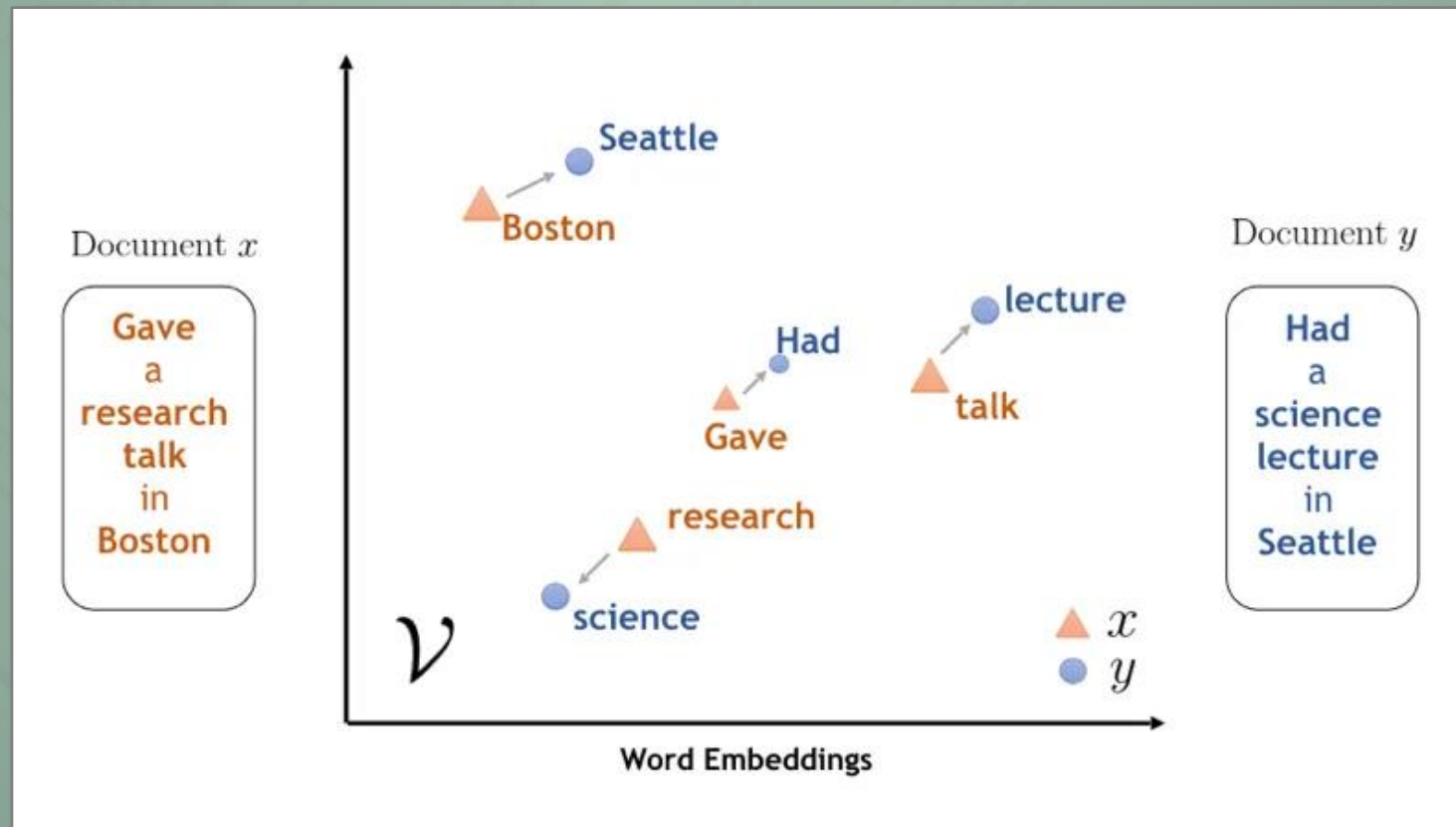
Example: BERT

- Bidirectional Encoder Representations from Transformers
- Text Classifier based on Attention mechanism



Contextualized Word Embedding

... allows you to calculate contextual distances of sentences, paragraphs, etc.



Comparison Table for Text Representation Models

Part 1

Approach	Primary Function	Description	Use Cases	Best For	Challenges
Bag-of-Words	Word frequency counting	- Creates numerical vectors - where each dimension represents unique word and values indicate frequency - ignoring word order and grammar	- Document classification - Spam detection - Topic modeling - Simple text analysis	- Simple classification tasks - Quick prototyping - Small to medium vocabularies - Interpretable results	- Ignores word order - Fails to capture context - High dimensionality - Sparse vectors
N-Gram	Sequential word patterns	- Captures contiguous sequences of n words - to preserve local context - and word order information	- Language modeling - Text prediction - Autocomplete systems - Spell checking	- Sequential pattern recognition - Local context preservation - Simple language models - Predictive text	- Exponential vocabulary growth - Data sparsity issues - Limited context window - Storage requirements
TF-IDF	Term importance weighting	Measures word importance - by combining term frequency (how often a word appears in a document) - with inverse document frequency (how rare the word is across all documents)	- Information retrieval - Document ranking - Keyword extraction - Search engines	- Document similarity - Search relevance - Feature selection - Text mining	- No semantic understanding - Document length sensitivity - Assumes independence - Static representations

Comparison Table for Text Representation Models

Part 2

Approach	Primary Function	Description	Use Cases	Best For	Challenges
One-hot Encoding	Categorical representation	- Represents each word as a binary vector with a single 1 and all other positions as 0 - creating orthogonal word representations	- Simple text classification - Educational purposes - Baseline models - Small vocabulary tasks	- Basic NLP learning - Small datasets - Simple neural networks - Categorical features	- No semantic relationships - High dimensionality - Sparse representations - Memory inefficient
CBOW	Context-to-word prediction	- Neural network model - that predicts target word based on its surrounding context words - learning dense word representations	- Semantic search - Text classification - Feature extraction - Word similarity tasks	- Frequent word processing - Fast training - Smaller datasets - General embeddings	- Poor with rare words - Out-of-vocabulary issues - Static representations - Context averaging
Skip-Gram	Word-to-context prediction	- Neural network model that predicts surrounding context words given target word - excelling at capturing semantic relationships	- Word analogies - Semantic similarity - Recommendation systems - Transfer learning	- Rare word representation - Rich relationships - Large datasets - Semantic tasks	- Slower training - Computationally intensive - Out-of-vocabulary words - Static embeddings

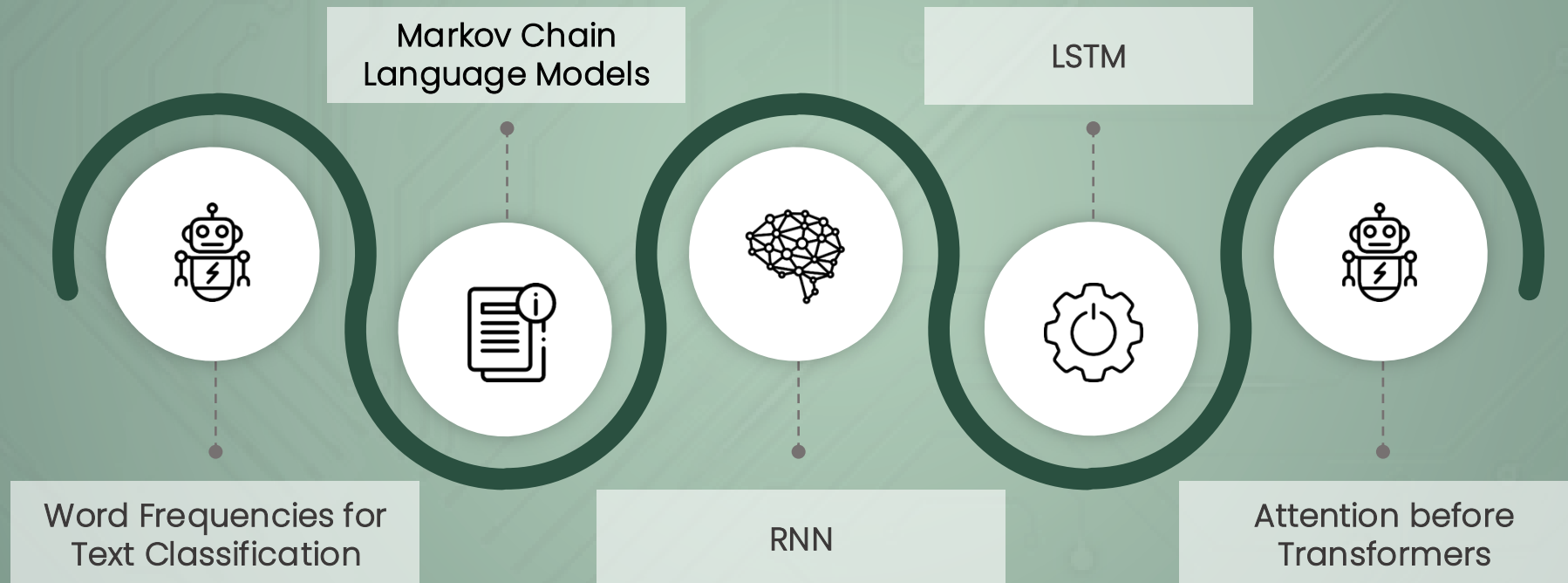
Comparison Table for Text Representation Models

Part 3

Approach	Primary Function	Description	Use Cases	Best For	Challenges
FastText	Subword-aware embeddings	Extends Word2Vec by representing - words as bags of character n-grams - enabling embeddings for unseen words - through subword information	- Morphologically rich languages - Social media text - Out-of-vocabulary handling - Multilingual applications	- Noisy text processing - Multiple languages - Unknown words - Morphological analysis	- Increased model size - More complex training - Subword noise - Parameter tuning
BERT	Contextualized embeddings	Bidirectional transformer model that - generates dynamic word representations - based on full sentence context - capturing nuanced meanings	- Question answering - Reading comprehension - Sentiment analysis - Language understanding	- Context-dependent tasks - High accuracy requirements - Complex NLP applications - Transfer learning	- Computational requirements - Large model size - Training complexity - Inference latency

Language Models

... are computational systems that predict the likelihood of sequences of words, enabling machines to understand, generate, and process human language effectively



Using Word Frequency for Text Classification

Count how often specific words appear in two books from different subjects (Physics & Biology) and use these frequencies as features for machine learning (e.g., logistic regression).

How It Works

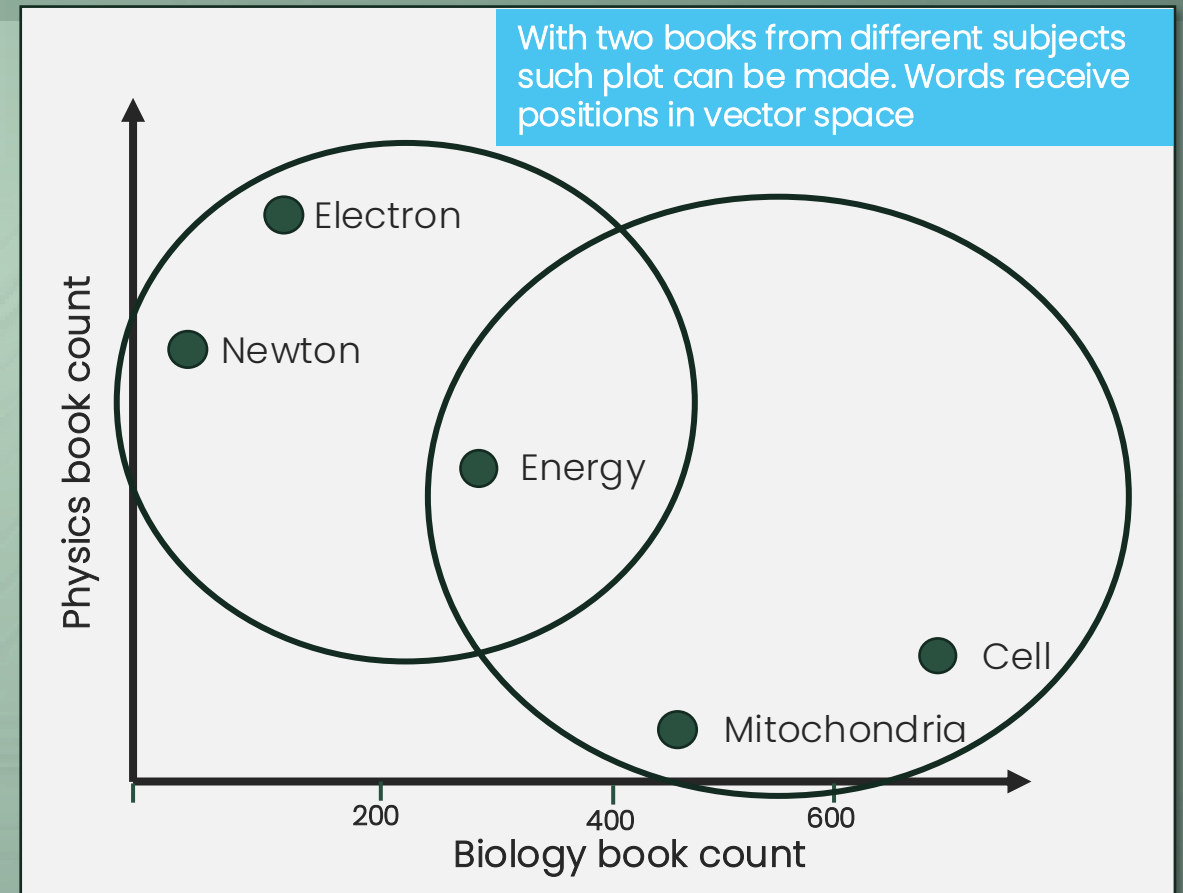
- Each point = a word (e.g., "Electron", "Cell")
- Words used mostly in physics (top left), mostly in biology (bottom right), or both (center)
- Circles show possible classification boundaries

Application

Use these word frequency features as input for

- Logistic regression
- Softmax (3 or more books/subjects)
- or similar models

to automatically classify new texts by subject.



Markov Chain Language Models

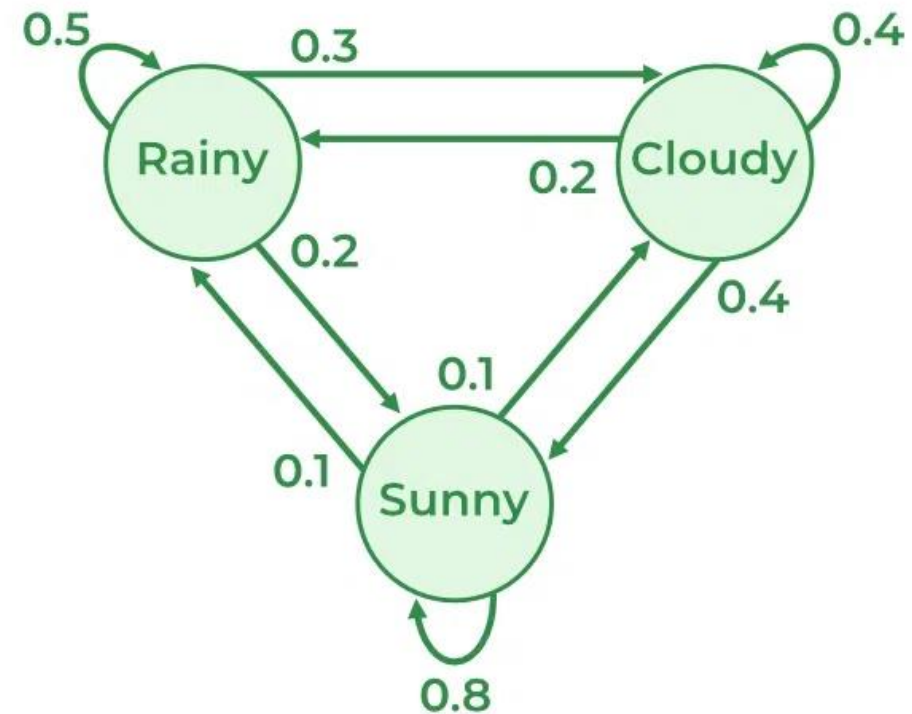
... are statistical models that predict next word in sentence based on current state (previous word or n words), assuming future states depend only on current state (Markov Property).

Key concepts

- States: Words or sequences of words.
- Transitions: Probabilities of moving from one state to another.
- Order: Defines how many previous states (words) are considered (e.g., first-order, second-order).

Example: First-Order Markov Model

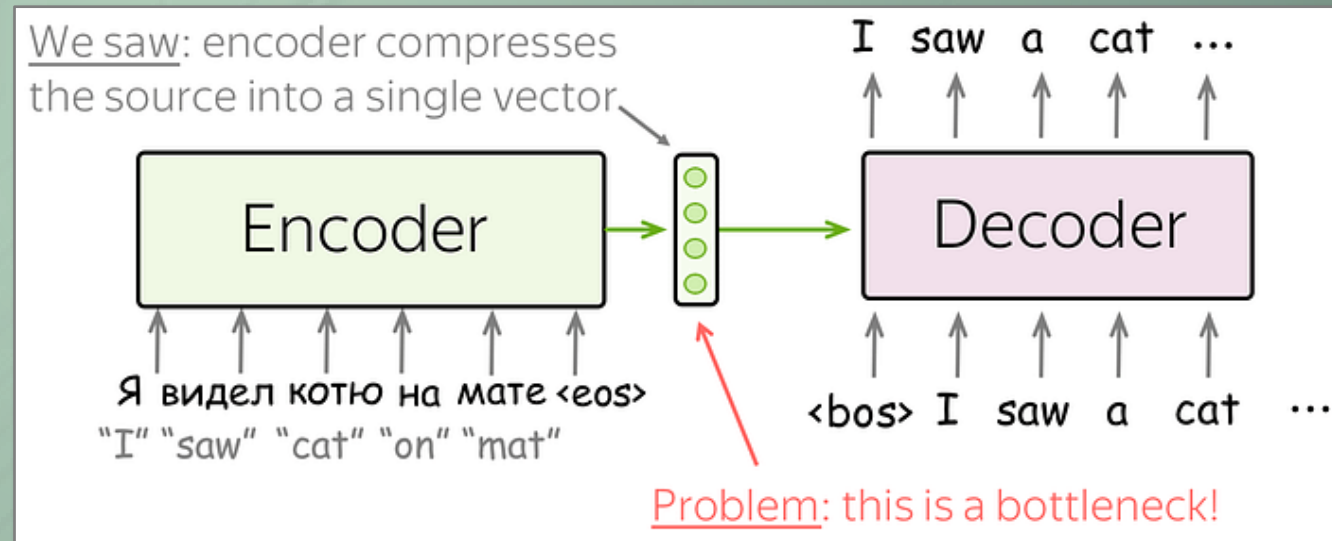
- Sentence: "I love natural language processing."
- Transitions: $P(\text{"love"} \mid \text{"I"})$, $P(\text{"natural"} \mid \text{"love"})$, $P(\text{"language"} \mid \text{"natural"})$, etc.



Recurrent neural nets (RNN)

... are neural networks designed for sequential data, using loops to persist information but struggling with long-term dependencies due to the vanishing gradient problem.

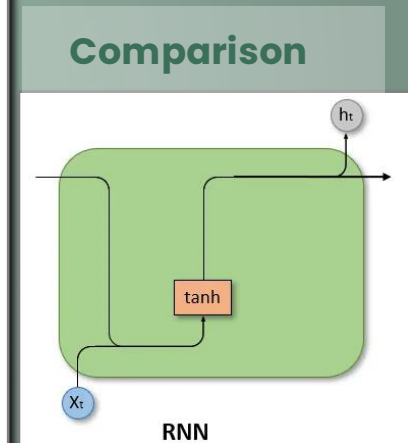
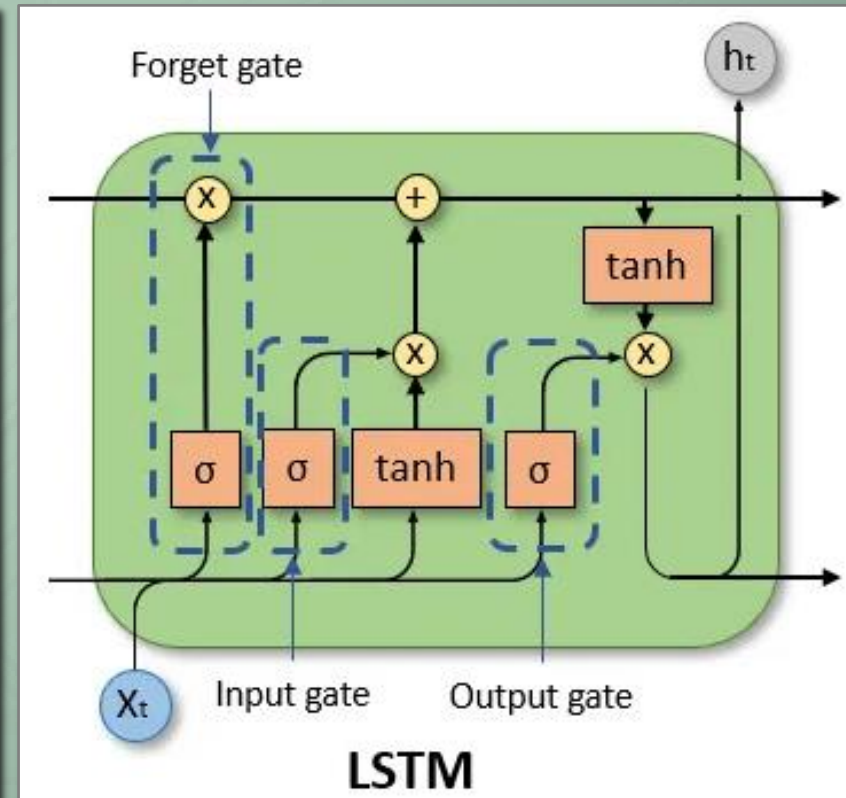
- Designed for sequential data processing.
- **Memory of previous inputs through loops:**
Each neuron is connected to itself
- Effective when order and context are important (e.g., language modeling, time-series prediction).
- Relatively easy to train and low computational intensity
- **Suffers from vanishing gradient problem:**
difficult to learn long-term dependencies.
- **Applications:**
Simple text generation and time-series forecasting.



Long Short-Term Memory (LSTM)

... are advanced type of RNN that include mechanisms called gates to effectively manage and retain long-term dependencies, overcoming limitations of standard RNNs.

- Designed to overcome vanishing gradient problem
- Utilizes unique architecture with cell states and three types of gates: forget gate, input gate, and output gate.
- Capable of learning and retaining long-term dependencies.
- More computationally intensive to train than standard RNNs.
- Were widely used before Transformers in applications such as
 - text generation
 - speech recognition
 - machine translation
 - sentiment analysis,
 - time-series forecasting



RNNs and LSTMs – Comparison and Limitation

LSTM handles long sequences more effectively. Nevertheless context length remains limited

Feature	RNN	LSTM
Definition	Neural network for sequential data	Advanced RNN variant for long-term dependencies
Architecture	Simple structure with loops	Complex structure with cell states and gates
Memory Capability	Short-term memory, struggles with long sequences	Long-term memory, handles long sequences more effectively
Vanishing Gradient	Prone to vanishing gradient problem	Designed to mitigate vanishing gradient problem
Components	Neurons with recurrent connections	Cell state, forget gate, input gate, output gate
Training Complexity	Easier to train, less computationally intensive	More complex to train, computationally intensive
Applications	Basic sequence modeling, time-series prediction	Text generation, speech recognition, time-series forecasting



Attention mechanism before “Transformers (2017)”

Attention mechanisms were a critical step in advancing natural language processing and understanding before the advent of Transformers

Bahdanau Attention

Neural Machine Translation by Jointly Learning to Align and Translate, Bahdanau, et al., 2014

- Introduced dynamic focus on different parts of the input sequence using additive scoring for neural machine translation.

Luong Attention

Effective Approaches to Attention-based Neural Machine Translation, Luong et al., 2015

- Improved computational efficiency with dot-product and general scoring functions for alignment in sequence-to-sequence models.

Self-Attention

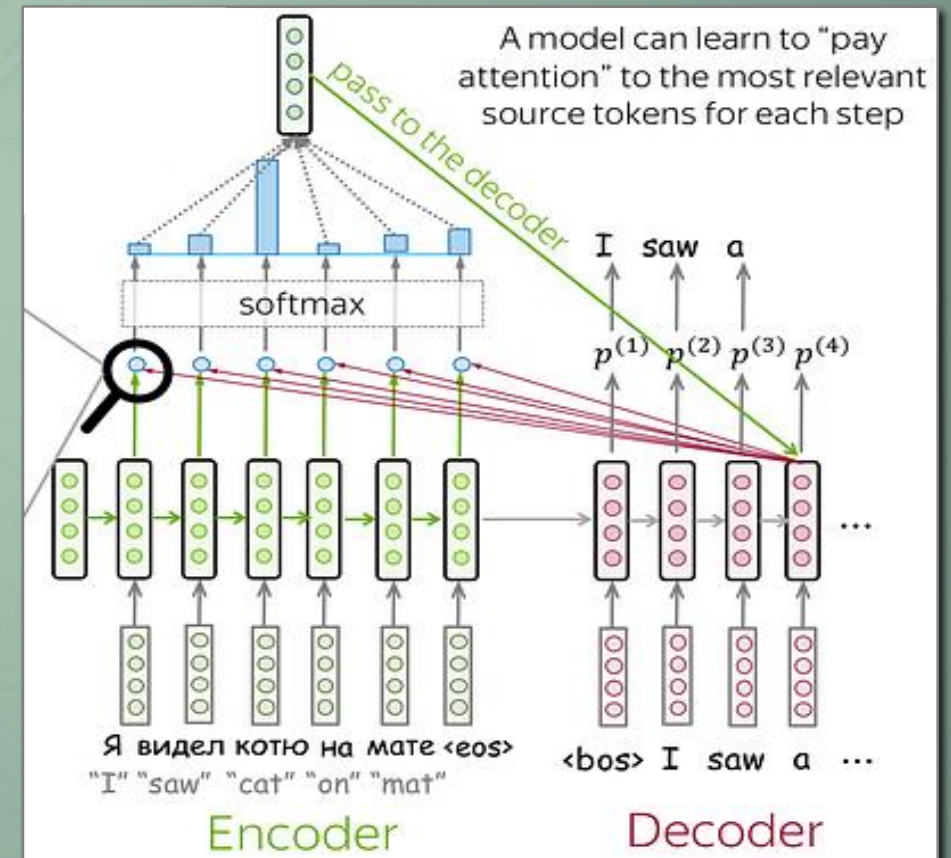
Long Short-Term Memory-Networks for Machine Reading, Cheng et al., 2016

- Enabled parallel computation by allowing each word to attend to all other words, forming the basis of the Transformer architecture.

Hierarchical Attention

Hierarchical Attention Networks for Document Classification, Yanh et al., 2016

- Applied attention at multiple levels (word and sentence) to better capture context in hierarchical data structures.



Comparison Table for Language Models

Part 1

Approach	Primary Function	Description	Use Cases	Best For	Challenges
Markov Chain Language Models	Statistical sequence prediction	Predicts next word ▪ based on n previous words ▪ using probability tables	▪ Text generationAutocomple ▪ Simple chatbots	▪ Fast inference ▪ Interpretable ▪ Small data	▪ Limited context ▪ Data sparsity ▪ No semantics
RNN	Sequential processing	Processes sequences ▪ with recurrent connections ▪ maintaining hidden state	▪ Time series ▪ Simple text generation ▪ Sequential classification	▪ Variable lengths ▪ Memory efficient ▪ Online processing	▪ Vanishing gradients ▪ Short memory ▪ Sequential bottleneck
LSTM	Long-term dependencies	Advanced RNN with gates ▪ to control information flow ▪ and memory	▪ Machine translation ▪ Speech recognition ▪ Sentiment analysis	▪ Long sequences ▪ Complex patterns ▪ Stable training	▪ Computational cost ▪ Still sequential ▪ Limited context

Comparison Table for Language Models

Part 2

Approach	Primary Function	Description	Use Cases	Best For	Challenges
Bahdanau Attention	Dynamic input focus	- Additive attention mechanism - for sequence-to-sequence translation	- Neural translation - Encoder-decoder tasks - Alignment problems	- Translation quality - Interpretability - Variable alignment	- Computational overhead - Quadratic complexity - Training difficulty
Luong Attention	Efficient alignment	- Dot-product attention - with simpler computation than Bahdanau	- Machine translation - Text summarization - Sequence modeling	- Computational efficiency - Implementation ease - Good performance	- Less flexible - Limited variants - Alignment constraints
Self-Attention	Parallel sequence processing	Each position attends to all positions in the sequence simultaneously	- Document classification - Language modeling - Transformer base	- Parallelization - Long dependencies - No recurrence	- Memory requirements - Quadratic scaling - Position encoding
Hierarchical Attention	Multi-level attention	Applies attention at multiple levels (word, sentence, document)	- Document classification - Long text analysis - Structured data	- Hierarchical data - Long documents - Interpretable structure	- Complex architecture - Multiple parameters - Training complexity

Links

i.e. sources for self-learning

	Title	Link
Basics of Natural Language Processing	An Introduction to Bag of Words (BoW) What is Bag of Words?	An Introduction to Bag of Words (BoW) What is Bag of Words?
	Understanding TF-IDF: A Traditional Approach to Feature Extraction in NLP	https://towardsdatascience.com/understanding-tf-idf-a-traditional-approach-to-feature-extraction-in-nlp-a5bfbe04723f
	Word Embedding and Word2Vec, Clearly Explained!!!	https://www.youtube.com/watch?v=viZrOnJcIY0
	Word2Vec Explained	https://towardsdatascience.com/word2vec-explained-49c52b4ccb71
	Word2vec : NLP & Word Embedding	https://datascientest.com/de/word2vec
	Stanford XCS224U: NLU I Intro & Evolution of Natural Language Understanding	https://www.youtube.com/watch?v=K_Dh0Sxujuc&list=PLoROMvodv4rOwvldxftJTmoR3kRcWkJBp
	Introduction NLP Tutorial For Beginners In Python	https://www.youtube.com/watch?v=R-AG4-qZs1A&list=PLeo1K3hjS3uuvuAXhYjV2IMEShq2UYSwX



About me

Dr. Harald Stein

- Data Scientist ~ 9 years experience
- AlgoTrader ~ 4 years experience
- Ph.D. in Economics, Game Theory
- LinkedIn: <https://www.linkedin.com/in/harald-stein-phd-1648b51a>
- ResearchGate: <https://www.researchgate.net/profile/Harald-Stein>

